



深度之眼
deepshare.net

LLM量化与推理

Adam老师

2025/10/11



目录

- LLM全参微调
- LLM量化
- LLM线上推理逻辑（生成式方法）
- Q&A



LLM全参微调

CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7

swift sft \

--model 'pretrained_models/qwen2.5-14b/' \

--train_type full \

--dataset './datasets/gen_cls/train_split_message.jsonl' \

--val_dataset './datasets/gen_cls/valid_split_message.jsonl' \

--torch_dtype bfloat16 \

--num_train_epochs \${EPOCHS} \

--per_device_train_batch_size 1 \

--per_device_eval_batch_size 1 \

--gradient_accumulation_steps 16 \

--learning_rate 1e-5 \

--eval_strategy 'epoch' \

--save_strategy 'epoch' \

--logging_steps 50 \

--max_length 1024 \

--output_dir \${OUTPUT_DIR} \

--system 'You are a helpful assistant.' \

--warmup_ratio 0.05 \

--dataloader_num_workers 4 \

--deepspeed zero2

LLM量化

- 为何需要量化：
 - 模型文件太大，无法上传到kaggle（单个model/dataset不能超过20G）
 - 模型文件太大，无法加载到GPU进行推理
- 量化方式：
 - 静态量化：fp8, int4, int8, NF4
 - 动态量化：AWQ, GPTQ

方式	特点	是否需要数据集	举例
纯参数量化 / 非感知PTQ	直接对模型权重进行低精度表示（比如 float32 → int8），不看输入数据分布	❌ 不需要	你在一些开源仓库直接下载 <code>model.int8()</code> 、 <code>bitsandbytes</code> 量化模型
数据感知量化 / 校准型PTQ	除了量化权重，还会对激活（activation）进行量化，需要统计激活分布，避免精度大幅下降	✅ 需要校准数据集	<code>transformers</code> 的 <code>quantization_config</code> 里，如果选择 <code>calib_dataset</code> 就需要提供样本进行校准

LLM量化：静态量化

- 通过 transformers加载：支持各种静态量化类型
- 通过vllm加载：支持部分，比如 fp8、int8

```
: base_model_path = '/kaggle/input/qwen2.5/transformers/0.5b/1/'  
lora_path = '/kaggle/input/map_qwen25_p5b_cls/pytorch/default/3/qwen25_p5b_cls/'
```

```
: device = torch.device('cuda:0')
```

```
: bnb_config = BitsAndBytesConfig(  
    load_in_4bit=True,  
    bnb_4bit_use_double_quant=True,  
    bnb_4bit_quant_type="nf4",  
    bnb_4bit_compute_dtype=torch.bfloat16  
)
```

```
: base_model = AutoModel.from_pretrained(  
    base_model_path,  
    quantization_config=bnb_config,  
    device_map=device,  
)
```




LLM量化：动态量化

- 需要预量化，使用 GPTQModel 库，详细见脚本 do_gptq.py
- 推理时通过vllm加载

```
:  
import os  
os.environ["CUDA_VISIBLE_DEVICES"] = "0,1"  
  
model_name = "/kaggle/input/map_gen_cls_32b_full_qptq/pytorch/default/1/gen_cls_32b_full_qptq_4bit/"
```

```
:  
from vllm import LLM, SamplingParams  
llm = LLM(  
    model=model_name,  
    # quantization="gptq",  
    tensor_parallel_size=2,  
    gpu_memory_utilization=0.99,  
    trust_remote_code=True,  
    dtype="float16",  
    max_model_len=1500,  
    enforce_eager=True,  
    # max_num_seq = 30  
)  
tokenizer = llm.get_tokenizer()
```




LLM线上推理逻辑

- 详见: <https://www.kaggle.com/code/jiazhuang/qwen3-32b-gptq-infer>
- 技术点:
 - 大模型推理逻辑
 - 生成式模型如何做分类任务
 - 如果获取预测token的概率分布

大模型推理逻辑

三、Prefill 阶段（初始化上下文）

假设输入 prompt 是：

```
arduino  
  
"Hello, my name is"
```

经过 tokenizer 编码后，得到 token 序列：

```
csharp  
  
[x1, x2, x3, x4]
```

1 每一层的基本计算流程

每层 Decoder Block 包含：

- Self-Attention 子层
- Feed-Forward 子层 (MLP)
- LayerNorm 与残差连接

2 Masked Self-Attention

因为是 **Decoder-only**，所以注意力是“单向的”：

每个位置只能看到它之前的 token。

所以注意力 mask 长这样：

```
vbnet  
  
token_1: attends to [1]  
token_2: attends to [1,2]  
token_3: attends to [1,2,3]  
...
```

3 输出最后一个 token 的隐状态

模型在 prefill 阶段最终只关心 最后一个 token 的隐状态（比如 h_4 ），
通过一个 **Linear + Softmax** 层预测下一个 token：

$$P(x_5) = \text{softmax}(W_{out} \cdot h_4)$$



生成式模型如何做分类任务

- 类别少时使用数字或字母做索引，类别多时使用特殊符号
- 计算特殊符号的概率分布



获取预测token的概率分布 (transformers)

```
: model_inputs = tokenizer([text], return_tensors="pt").to(peft_model.device)

generated_ids = peft_model.generate(
    **model_inputs,
    max_new_tokens=1,
    output_scores=True,
    return_dict_in_generate=True
)
```

```
: for t, i in zip(label_map_df.token, allowed_token_ids):
    print(t, i, generated_ids.scores[0][0][i])
```

```
→ 51018 tensor(-inf, device='cuda:1')
← 71858 tensor(-inf, device='cuda:1')
↑ 76286 tensor(-inf, device='cuda:1')
↓ 79029 tensor(-inf, device='cuda:1')
↔ 145023 tensor(-inf, device='cuda:1')
‡ 146853 tensor(-inf, device='cuda:1')
< 144138 tensor(31.9728, device='cuda:1')
> 144129 tensor(-inf, device='cuda:1')
『 43520 tensor(-inf, device='cuda:1')
』 35661 tensor(-inf, device='cuda:1')
| 72887 tensor(-inf, device='cuda:1')
- 14555 tensor(-inf, device='cuda:1')
┌ 145430 tensor(-inf, device='cuda:1')
└ 145624 tensor(-inf, device='cuda:1')
┌ 144798 tensor(-inf, device='cuda:1')
└ 145474 tensor(-inf, device='cuda:1')
+ 146529 tensor(-inf, device='cuda:1')
■ 15199 tensor(-inf, device='cuda:1')
⊗ 145326 tensor(-inf, device='cuda:1')
```


获取预测token的概率分布 (vllm)

函数签名

python

复制代码

```
def __call__(self, input_ids: torch.LongTensor, scores: torch.FloatTensor) -> torch.FloatTensor
```

1. input_ids

- 形状: [batch_size, seq_len]
- 内容: 当前已经生成的序列 (prompt + 已生成 token)。
- 每个元素是一个 token 的 id。
- 例子:

python

复制代码

```
input_ids = tensor([[15496, 11, 616, 3290]])  
# => 对应文本 "Hello, my favorite"
```

2. scores

- 形状: [batch_size, vocab_size]
- 内容: 模型在当前这一步预测的 logits (未归一化概率)。
 - 每一行是一个样本对应的所有词表 token 的分数。
- 默认情况下, 下一步 softmax 后就是模型生成 token 的概率分布。

```
allowed_token_ids = [tokenizer.encode(str(i), add_special_tokens=False)[0] for i in special_characters_list]
```

```
from transformers import LogitsProcessor
```

```
class LabelOnlyLogitsProcessor(LogitsProcessor):
```

```
    def __init__(self, allowed_token_ids):
```

```
        self.allowed_token_ids = allowed_token_ids
```

```
    def __call__(self, input_ids: torch.Tensor, scores: torch.Tensor) -> torch.Tensor:
```

```
        mask = torch.full_like(scores, float('-inf'))
```

```
        if scores.dim() == 1:
```

```
            mask[self.allowed_token_ids] = 0
```

```
        elif scores.dim() == 2:
```

```
            mask[:, self.allowed_token_ids] = 0
```

```
        else:
```

```
            raise ValueError("Unexpected score dimensions")
```

```
        return scores + mask
```

```
sampling_params = SamplingParams(
```

```
    # temperature=0.7,
```

```
    temperature=0,
```

```
    max_tokens=1,
```

```
    logprobs=8,
```

```
    stop=["\n", "."],
```

```
    logits_processors=[LabelOnlyLogitsProcessor(allowed_token_ids)],
```

```
)
```


Q&A

